

C++ Library Fundamentals TS Issues to be moved in Kona

Doc. no. P2704R0

Date: 2022-11-07

Audience: WG21

Reply to: Jonathan Wakely <lwgchair@gmail.com>

Tentatively Ready Issues

[3136](#). [fund.ts.v3] LFTSv3 awkward wording in propagate_const requirements

Section: 3.2.2.2.1 [fund.ts.v3::propagate_const.class_type_requirements] **Status:** [Tentatively Ready](#) **Submitter:** Thomas Köppe **Opened:** 2018-07-02 **Last modified:** 2022-10-19

Priority: 3

Discussion:

Addresses: fund.ts.v3

In the LFTSv3 prospective-working-paper [N4758](#), [propagate_const.class_type_requirements] uses a strange turn of phrase:

"In this sub-clause, t denotes a non-const lvalue of type T , ct is a const $T\&$ bound to t , [...]"

The last bit is strange: " ct is a const $T\&$ bound to t " is not how we usually say things. The specification-variables usually denote values, and values can't be references. Perhaps we could just say, " ct is `as_const(t)`"?

[2018-07-20 Priority set to 3 after reflector discussion]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4758](#).

1. Edit 3.2.2.2.1 [fund.ts.v3::propagate_const.class_type_requirements] as indicated:

-1- If T is class type then it shall satisfy the following requirements. In this sub-clause t denotes a non-const lvalue of type T , ct is ~~a const $T\&$ bound to t~~ `as_const(t)`, `element_type` denotes an object type.

[2022-10-12; Jonathan provides improved wording]

[2022-10-19; Reflector poll]

Set status to "Tentatively Ready" after eight votes in favour in reflector poll.

Proposed resolution:

This wording is relative to [N4920](#).

1. Edit 3.2.2.2 [fund.ts.v3::propagate_const.requirements] as indicated:

-1- τ shall be a cv-unqualified pointer-to-object type, an object pointer type or a cv-unqualified class type for which `decltype(*declval<T&>())` is an lvalue reference to object type; otherwise the program is ill-formed.

~~-2- If τ is an array type, reference type, pointer to function type or pointer to (possibly cv-qualified) void, then the program is ill-formed.~~

-3- [Note: `propagate_const<const int*>` is well-formed but `propagate_const<int* const>` is not. — end note]

2. Edit 3.2.2.2.1 [fund.ts.v3::propagate_const.class_type_requirements] as indicated:

-1- If τ is class type then it shall satisfy the following requirements. In this sub-clause t denotes a non-const lvalue of type τ , ct is a const T bound to t , $element_type$ denotes an object type. `denotes as const(t).`

3411. [fund.ts.v3] Contradictory namespace rules in the Library Fundamentals TS

Section: 5.4 [fund.ts.v3::memory.resource.syn] **Status:** [Tentatively Ready](#) **Submitter:** Thomas Köppe **Opened:** 2020-02-28
Last modified: 2022-10-19

Priority: 3

Discussion:

Addresses: fund.ts.v3

The Library Fundamentals TS, [N4840](#), contains [a rule about the use of namespaces \(paragraph 1\)](#), with the consequence:

"This TS does not define `std::experimental::fundamentals_v3::pmr`"

However, the TS then [goes on to define exactly that namespace](#).

At the time when the subclause `memory.resource.syn` was added, the IS didn't use to contain a namespace `pmr`. When the IS adopted that namespace and the TS was rebased, the namespace rule started conflicting with the material in the TS.

I do not have a workable proposed resolution at this point.

[2020-04-07 Issue Prioritization]

Priority to 3 after reflector discussion.

[2021-11-17; Jiang An comments and provides wording]

Given namespaces `std::chrono::experimental::fundamentals_v2` and `std::experimental::fundamentals_v2::pmr` are used in LFTS v2, I think that the intent is that

- the prefix `experimental::fundamentals_vN` is only added to a namespace specified by the standard, and
- every name specified by the standard in the outer namespace of that of the related component in the TS.

If we follow the convention, perhaps we should relocate `resource_adaptor` from `std::experimental::fundamentals_v3::pmr` to `std::pmr::experimental::fundamentals_v3` in LFTS v3. If it's decided that `resource_adaptor` shouldn't be relocated, I suppose that LWG 3411 can be by striking the wrong wording in 1.3 [fund.ts.v3::general.namespaces] and using qualified `std::pmr::memory_resource` when needed.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4853](#).

[Drafting Note: Two mutually exclusive options are prepared, depicted below by **Option A** and **Option B**, respectively.]

Option A:

1. Modify 1.3 [fund.ts.v3::general.namespaces] as indicated:

-2- Each header described in this technical specification shall import the contents of `outer-namespace` `std::experimental::fundamentals_v3` into `outer-namespace` `std::experimental` as if by

```
namespace outer-namespace std::experimental::inline fundamentals_v3 {}
```

where `outer-namespace` is a namespace defined in the C++ Standard Library.

2. Modify 5.3 [fund.ts.v3::memory.type.erased allocator], Table 5, as indicated:

Table 5 — Computed memory_resource for type-erased allocator

If the type of alloc is	then the value of rptr is
	[...]
any other type meeting the requirements (C++20 ¶16.5.3.5)	a pointer to a value of type <code>pmr::experimental::resource_adaptor<A></code> where A is the type of alloc. rptr remains valid only for the lifetime of x.
	[...]

3. Modify 5.4 [fund.ts.v3::memory.resource.syn], header <experimental/memory_resource> synopsis, as indicated:

```
namespace std::pmr::experimental::inline fundamentals_v3::pmr {  
[...]  
} // namespace std::pmr::experimental::inline fundamentals_v3::pmr
```

Option B:

1. Modify 1.3 [fund.ts.v3::general.namespaces] as indicated:

-1- Since the extensions described in this technical specification are experimental and not part of the C++ standard library, they should not be declared directly within namespace `std`. Unless otherwise specified, all components described in this technical specification either:

— modify an existing interface in the C++ Standard Library in-place,

— are declared in a namespace whose name appends `::experimental::fundamentals_v3` to a namespace defined in the C++ Standard Library, such as `std` or `std::chrono`, or

— are declared in a subnamespace of a namespace described in the previous bullet, whose name is not the same as an existing subnamespace of namespace `std`.

[Example: This TS does not define `std::experimental::fundamentals_v3::pmr` because the C++ Standard Library defines `std::pmr`. — end example]

2. Modify 4.2 [fund.ts.v3::func.wrap.func], class template function synopsis, as indicated:

```
namespace std {  
    namespace experimental::inline fundamentals_v3 {  
  
        template<class> class function; // undefined
```

```

template<class R, class... ArgTypes>
class function<R(ArgTypes...)> {
public:
    [...]
    std::pmr::memory_resource* get_memory_resource() const noexcept;
};
[...]
} // namespace experimental::inline fundamentals_v3
[...]
} // namespace std

```

3. Modify 5.3 [fund.ts.v3::memory.type.erased allocator], Table 5, as indicated:

Table 5 — Computed memory_resource for type-erased allocator

If the type of alloc is	then the value of rptr is
	[...]
any other type meeting the requirements (C++20 ¶16.5.3.5)	a pointer to a value of type experimental::pmr::resource_adaptor<A> where A is the type of alloc. rptr remains valid only for the lifetime of x.
	[...]

4. Modify 5.5.1 [fund.ts.v3::memory.resource.adaptor.overview] as indicated:

-1- An instance of resource_adaptor<Allocator> is an adaptor that wraps a **std::pmr::memory_resource** interface around Allocator. [...]

```

// The name resource_adaptor_imp is for exposition only.
template<class Allocator>
class resource_adaptor_imp : public std::pmr::memory_resource {
public:
    [...]

    virtual bool do_is_equal(const std::pmr::memory_resource& other) const noexcept;
};

```

5. Modify 5.5.3 [fund.ts.v3::memory.resource.adaptor.mem] as indicated:

-6- bool do_is_equal(const **std::pmr::memory_resource**& other) const noexcept;

[...]

6. Modify 7.2 [fund.ts.v3::futures.promise], class template promise synopsis, as indicated:

```

namespace std {
    namespace experimental::inline fundamentals_v3 {

        template<class R>
        class promise {
        public:
            [...]
            std::pmr::memory_resource* get_memory_resource() const noexcept;
        };
    }
} // namespace experimental::inline fundamentals_v3
[...]
} // namespace std

```

7. Modify 7.3 [fund.ts.v3::futures.task], class template packaged_task synopsis, as indicated:

```

namespace std {
    namespace experimental::inline fundamentals_v3 {

        template<class R, class... ArgTypes>
        class packaged_task<R(ArgTypes...)> {

```

```

    public:
    [...]
    std::pmr::memory_resource* get_memory_resource() const noexcept;
};
[...]
} // namespace experimental::inline fundamentals_v3
[...]
} // namespace std

```

[2022-10-12; Jonathan provides updated wording]

The LWG telecon decided on a simpler form of **Option A**. The changes to 1.3 [fund.ts.v3::general.namespaces] generated some questions and disagreement, but it was decided that they are not needed anyway. The normative synopses already depict the use of inline namespaces with the stated effects. That paragraph seems more informative than normative, and there were suggestions to strike it entirely. It was decided to keep it but without making the edits. As such, it remains correct for the contents of `std::experimental::fundamentals_v3`. It doesn't apply to `pmr::resource_adaptor`, but is not *incorrect* for that either. The rest of the proposed resolution fully specifies the `pmr` parts.

[2022-10-19; Reflector poll]

Set status to "Tentatively Ready" after eight votes in favour in reflector poll.

Proposed resolution:

This wording is relative to [N4920](#).

1. Modify 5.3 [fund.ts.v3::memory.type.erased allocator], Table 5, as indicated:

Table 5 — Computed <code>memory_resource</code> for type-erased allocator	
If the type of <code>alloc</code> is	then the value of <code>rptr</code> is
	[...]
any other type meeting the requirements (C++20 ¶16.5.3.5)	a pointer to a value of type <code>pmr::experimental::resource_adaptor<A></code> where <code>A</code> is the type of <code>alloc</code> . <code>rptr</code> remains valid only for the lifetime of <code>x</code> .
	[...]

2. Modify 5.4 [fund.ts.v3::memory.resource.syn], header `<experimental/memory_resource>` synopsis, as indicated:

```

namespace std::pmr::experimental::inline fundamentals_v3::pmr {
[...]
} // namespace std::pmr::experimental::inline fundamentals_v3::pmr

```

[3771](#). [fund.ts.v3] remove binders typedefs from function

Section: 99 [fund.ts.v3::func.wrap.func.overview] **Status:** [Tentatively Ready](#) **Submitter:** Alisdair Meredith **Opened:** 2022-09-12 **Last modified:** 2022-09-23

Priority: Not Prioritized

Discussion:

Addresses: `fund.ts.v3`

The LFTSv3 bases its specification for `experimental::function` on `std::function` in the C++20 standard. However, the wording was largely copied over from LFTSv2 which based its wording on the C++14 standard.

For C++17, we removed the conditionally defined typedefs for the legacy binders API, but this removal was not reflected in the TS. We are now left with a specification referring to unknown types, `T1` and `T2`.

These typedefs should be excised to match the referenced standard.

[2022-09-23; Reflector poll]

Set status to Tentatively Ready after ten votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4920](#).

1. Modify the synopsis in 99 [fund.ts.v3::func.wrap.func.overview] as indicated:

```
namespace std {
    namespace experimental::inline fundamentals_v3 {

        template<class> class function; // undefined

        template<class R, class... ArgTypes>
        class function<R(ArgTypes...)> {
        public:
            using result_type = R;
            using argument_type = T1;
            using first_argument_type T1;
            using second_argument_type = T2;

            using allocator_type = erased_type;

            // ...
        }
    }
```